==========

==========

CHAPTER 24 — BOOT PROCESS & MMU BRING-UP (EL3 → EL2 → EL1)

==========

==========

A maximum-detail, RTOS/QNX-style breakdown of how ARMv8 systems boot, how each exception level configures the MMU, how early page tables are built, and how control transitions from EL3 → EL2 → EL1 → EL0.

This chapter explains **exactly how real systems bring up the MMU**, including secure boot, hypervisor initialization, kernel entry, and early memory mapping.

****

****

24.1 — Why MMU Bring-Up Is Critical

****

****

MMU bring-up determines:

- system security
- system stability
- early boot correctness
- interrupt vector mapping
- device access correctness
- hypervisor isolation
- kernel memory layout

A mistake here → system crashes before the OS even boots.

****

****

24.2 — ARMv8 Boot Flow Overview

****

****

The standard ARMv8 boot sequence:

```
Reset → EL3 (Secure Monitor)
      → EL2 (Hypervisor)
      → EL1 (Kernel/OS)
      → EL0 (User Space)
```

Each level configures **its own MMU context**.

****

****

24.3 — EL3: Secure Monitor Boot Responsibilities

****

****

EL3 is responsible for:

- secure boot
- TrustZone configuration
- secure/non-secure memory partitioning
- initial identity mapping
- enabling MMU for secure world
- preparing EL2 entry

EL3 Must Map:

- secure RAM
- secure MMIO
- exception vectors
- EL2 entry code

EL3 MMU Requirements:

- identity map early memory
- use Device-nGnRnE for MMIO
- use Normal WB for RAM
- enable SCTLR_EL3.M

****

****

24.4 — EL3 MMU Bring-Up Sequence

****

****

Typical EL3 MMU enable sequence:

```
1. Build early identity page tables
2. Write TTBR0_EL3 with table base
3. Configure TCR_EL3 (granule, shareability, cacheability)
4. Configure MAIR_EL3
5. DSB SY
6. ISB
7. Enable MMU: SCTLR_EL3.M = 1
```

Why identity mapping?

Because before the MMU is enabled, PC and SP use physical addresses.

****

****

24.5 — Transition from EL3 → EL2

****

****

EL3 prepares EL2 by:

- setting SCR_EL3.NS = 1 (non-secure world)
- setting HCR_EL2 (virtualization controls)
- passing boot parameters
- mapping EL2 code/data
- switching to non-secure EL2

Transition Instruction:

```
ERET
```

****

****

24.6 — EL2: Hypervisor Boot Responsibilities

****

****

EL2 is responsible for:

- Stage-2 MMU bring-up
- VMID allocation
- IPA → PA mapping
- GIC virtualization
- timer virtualization
- device virtualization
- preparing EL1 entry

EL2 Must Map:

- hypervisor code/data
- Stage-2 page tables
- GIC MMIO
- timer MMIO
- guest RAM (IPA → PA)

****

****

24.7 — EL2 MMU Bring-Up Sequence

****

****

Typical EL2 MMU enable sequence:

1. Build Stage-2 page tables
2. Write TTBR0_EL2 with S2 table base
3. Configure TCR_EL2 (IPA size, granule)
4. Configure MAIR_EL2
5. Configure HCR_EL2 (virtualization)
6. DSB SY
7. ISB
8. Enable MMU: SCTLR_EL2.M = 1

Stage-2 must be enabled before launching EL1 guest OS.

****

****

24.8 — Transition from EL2 → EL1

****

****

EL2 prepares EL1 by:

- setting HCR_EL2 to trap or allow features
- setting VTTBR_EL2 for guest Stage-2
- mapping EL1 kernel entry point
- setting SPSR_EL2 (EL1 mode)
- setting ELR_EL2 (EL1 entry address)

Transition Instruction:

ERET

****

****

24.9 — EL1: Kernel Boot Responsibilities

****

****

EL1 is responsible for:

- Stage-1 MMU bring-up
- TTBR0/TTBR1 setup
- ASID allocation
- kernel memory layout
- device mapping
- interrupt vector mapping
- enabling caches
- enabling SMP

EL1 Must Map:

- kernel code/data
- kernel stacks
- exception vectors
- device MMIO
- user space regions (later)

****

****

24.10 — EL1 MMU Bring-Up Sequence

****

****

Typical EL1 MMU enable sequence:

1. Build kernel page tables (TTBR1)
2. Build early user page tables (TTBR0)
3. Write TTBR1_EL1 with kernel table base
4. Write TTBR0_EL1 with user table base
5. Configure TCR_EL1 (granule, shareability, cacheability)
6. Configure MAIR_EL1
7. DSB SY
8. ISB
9. Enable MMU: SCTLR_EL1.M = 1

Kernel uses TTBR1_EL1

User space uses TTBR0_EL1

****

****

24.11 — EL1 Identity Mapping Requirements

****

****

During early boot, EL1 must identity-map:

- kernel entry code
- early stack
- exception vectors
- device MMIO used during boot

Once MMU is enabled, kernel switches to **virtual addresses**.

****

****

24.12 — Transition from EL1 → EL0

****

****

EL1 prepares EL0 by:

- building user page tables
- assigning ASID
- mapping user code/data/stack
- setting SPSR_EL1 (EL0 mode)
- setting ELR_EL1 (user entry point)

Transition Instruction:

Code
ERET

****

****

24.13 — Early Page Table Design

****

****

Early page tables must be:

- identity-mapped
- minimal
- contiguous
- aligned
- using blocks where possible
- using pages for MMIO

Typical Early Layout

Code

```
0x0000_0000 → 0x3FFF_FFFF   RAM (1 GB blocks)
0x4000_0000 → 0x4FFF_FFFF   MMIO (4 KB pages)
```

****

****

24.14 — Early Device Mapping

****

****

Devices required during boot:

- UART
- GIC
- timers
- watchdog
- debug ports

Must be mapped:

- Device-nGnRnE
- Non-Shareable
- 4 KB pages
- AP=RW
- PXN/UXN

****

****

24.15 — Early Interrupt Vector Mapping

****

****

Exception vectors must be:

- executable
- readable
- Normal WB
- Inner Shareable
- mapped in TTBR1_EL1

Common Mistake

Mapping vectors as Device memory → CPU cannot fetch instructions.

****

****

24.16 — Early Cache Enable Sequence

****

****

Caches must be enabled **after** MMU:

Code

```
1. Enable SCTLR_EL1.C (data cache)
2. Enable SCTLR_EL1.I (instruction cache)
3. ISB
```

Why?

Caches require virtual addresses.

****

****

24.17 — Early SMP Bring-Up

****

****

Secondary cores require:

- identical page tables
- identical TCR/MAIR
- identical TTBR1_EL1
- identical MMIO mappings

SMP Bring-Up Sequence

Code

```
1. Secondary core starts in EL3
2. EL3 → EL2 → EL1
3. Load same TTBR1_EL1
4. Enable MMU
5. Enable caches
6. Join SMP cluster
```

****

****

24.18 — Boot Failure Modes

****

****

Failure 1 — Wrong MAIR attributes

Device memory mapped as Normal → crash.

Failure 2 — Wrong TCR granule

Page tables unreadable → crash.

Failure 3 — Missing identity mapping

PC jumps to unmapped address → crash.

Failure 4 — Missing DSB/ISB

MMU not fully enabled → unpredictable behavior.

Failure 5 — Incorrect Stage-2 mapping

Guest kernel cannot boot.

****

****

24.19 — Full Boot Example (EL3 → EL2 → EL1 → EL0)

****

****

EL3

Code

```
Identity map early memory
Enable MMU
Configure TrustZone
ERET → EL2
```

EL2

Code

```
Build Stage-2 tables
Enable Stage-2 MMU
Configure virtualization
ERET → EL1
```

EL1

```
Code
Build kernel page tables
Enable Stage-1 MMU
Enable caches
Start kernel
ERET → EL0
```

EL0

```
Code
User program begins execution
```

****

****

24.20 — Chapter Summary

****

****

This chapter covered:

- full boot flow (EL3 → EL2 → EL1 → EL0)
- MMU bring-up at each exception level

- identity mapping
- early page-table design
- early device mapping
- early interrupt vector mapping
- early cache enable
- SMP bring-up
- failure modes
- full examples